TradingView Clone

Real-Time Market Monitoring and Trading Dashboard

Undergraduate Project Report

Submitted by: [Student Name]

Register Number: [Register Number]

Department: [Department Name]

Institution: [Institution Name]

Project Guide: [Guide Name]

Academic Year: [Academic Year]

Submission Date: [Submission Date]

Abstract

Financial-market information is distributed across data feeds, charts, personal watchlists, and discussion tools. This project addresses that fragmentation through a browser-based market-monitoring dashboard with a vanilla HTML, CSS, and JavaScript frontend, a Node.js and Express.js REST API, MongoDB persistence through Mongoose, and Binance public market data for supported cryptocurrency pairs. The backend includes account registration and login using bcryptjs password hashing and JWT bearer tokens, user-specific watchlist records, idea comments, configurable price alerts, in-app notifications, and market-data proxy endpoints. The chart client also connects to Binance WebSocket streams, while the backend alert monitor polls Binance prices at a configurable interval (30 seconds by default). The data model contains User, Watchlist, Alert, Comment, and Notification collections. Stock and index quotations are identified in the implementation as static or demo data; live stock alert monitoring is not configured. Source inspection confirms that the relevant routes, controllers, models, and browser modules are present, but a live deployment, database session, API response, and end-to-end behavior were not executed during report preparation. The project therefore demonstrates an integrated educational architecture and functional code paths without claiming measured performance or production readiness. The outcome is a modular full-stack prototype that illustrates client-server communication, authentication, persistence, and external market-data integration.

Table of Contents

Abstract 2
1. Introduction 3
2. Objectives 3
3. Literature Review / Background Study 4
4. System Design 4
4.1 System Architecture Diagram 5
4.2 Description of Modules and Key Features 5
5. Implementation 6
5.1 Technologies Used 6
5.2 Development Tools 7
5.3 Sample Code Snippets 7
6. Results and Output 8
6.1 Screenshots of the Working Project 8
6.2 Key Outcomes 9
7. Conclusion 9
8. Future Scope 9
9. Mapping to Program Outcomes (PO) and Sustainable Development Goals (SDG) 10
10. References 10

1. Introduction

Financial markets produce changing prices and large volumes of time-series data. A trader or learner may need to consult one interface for price charts, another for a personal symbol list, and separate tools for account preferences or discussion. This project builds a single educational dashboard that brings a charting workspace, market summary, watchlist, community ideas, and account-based utilities together in a browser.

Problem Statement

The project investigates how a lightweight web application can present cryptocurrency market data and provide user-scoped tools without depending on a frontend framework. The system must fetch or stream supported market values, retain account-owned records, and restrict private API operations to authenticated users. Its scope is monitoring and learning; it does not place exchange orders or provide investment recommendations.

Motivation and Importance

A clone-style project offers practical experience with client-side rendering, asynchronous HTTP and WebSocket communication, REST resource design, password hashing, token validation, document-oriented data modeling, and third-party API integration. The modules also make it possible to discuss what remains outside a prototype, such as live equity coverage, robust alert delivery, deployment operations, and security review.

Technology Rationale and Scope

Vanilla JavaScript keeps the browser layer close to web-platform fundamentals and avoids a React dependency. Node.js supports JavaScript across client and server contexts; Express organizes HTTP routes and middleware. MongoDB with Mongoose stores flexible user, alert, comment, and notification records. Binance public REST and WebSocket interfaces supply crypto data where supported. The repository contains static HTML pages, modular CSS and JavaScript files, and an ES-module Express backend. The source has no root-level frontend package.json; backend dependencies are defined in backend/package.json.

2. Objectives

Objective	Evidence-based scope
Interactive market monitoring	Browser dashboard/chart and crypto price updates through Binance interfaces. Source exists; runtime is not independently tested here.
Secure account access	Registration/login with bcryptjs password hashing, JWT bearer tokens, and protected routes.
Persistent watchlists	User assets in MongoDB, with compatibility for older documents containing a symbols array.
Community discussion	Read by idea identifier; authenticated users can post or delete their own comments.
Price alerts	CRUD for alerts; monitor evaluates crypto alerts only against Binance prices.
In-app notifications	Persist alert notifications; list, mark read, mark all read, and delete.
Market API access	Klines, 24h ticker and market status; stock/index values explicitly identified as demo/static.
Maintainable backend	Separated routes/controllers/models/config/middleware/service; validates environment variable names and JWT secret length.

3. Literature Review / Background Study

Market-monitoring platforms commonly combine chart display, symbol search, watchlists, news or community content, and alerting. TradingView describes charting and market-analysis capabilities in its product documentation [1]. This project uses those broad interaction patterns as an educational reference; it is not affiliated with or a replica of proprietary internals. A clone cannot guarantee the breadth, licensing, availability, or execution quality of a commercial platform. The implementation itself marks stocks and indices as static demo data, so its live-data scope is narrower than an all-market platform.

The browser uses standard HTML, CSS, and JavaScript and calls public market interfaces. Node.js provides a JavaScript runtime suited to network-oriented services [2]. Express maps HTTP methods and URLs to route handlers and middleware [3]. MongoDB stores records as documents, while Mongoose schemas define application-side fields and models provide a persistence interface [4]. These choices support a compact learning system but do not by themselves ensure availability, scale, or security.

JWT is a compact claims format for representing claims between parties; here it carries an account identifier and token version and is verified by middleware [5]. bcryptjs provides a JavaScript bcrypt implementation used to hash and compare passwords [6]. Binance developer documentation describes REST and streaming interfaces for programmatic market-data access [7]. The project consumes selected public endpoints and streams; it does not implement exchange order placement.

The educational value is the integration of these components and the explicit treatment of partial coverage. External live systems depend on network availability, provider limits, compatible symbols, and deployment configuration. Code-level capabilities are distinguished from runtime-tested results, and formal test and deployment work remains future scope.

4. System Design

The application follows a browser-client and REST-service structure. Frontend modules render the UI, maintain transient state, and call the API client. Express applies CORS/body parsing and route-level JWT protection where required. Controllers perform validation and persistence through Mongoose models. The app uses MongoDB collections named users, watchlist, alerts, comments, and notifications. Market data also travels directly from Binance WebSocket streams to the browser chart and watchlist ticker; REST klines/ticker calls are proxied through Express. A periodic backend monitor reads active alerts and checks Binance prices for crypto symbols.

4.1 System Architecture Diagram

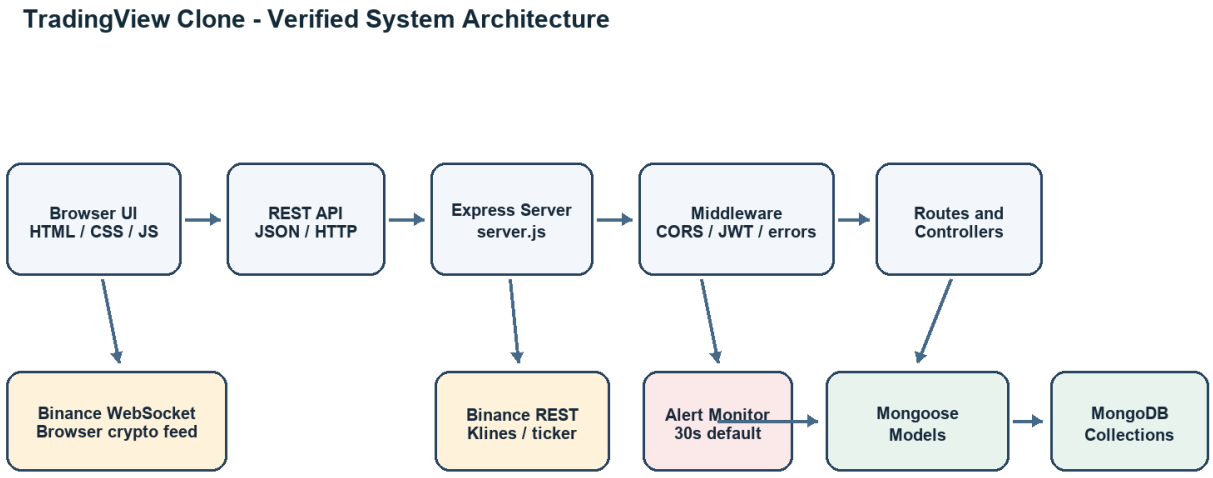


Figure 1. Architecture based on server, routes, models, market controllers, and client WebSocket modules.

The diagram distinguishes public REST market requests through the Express proxy from direct browser WebSocket connections for live crypto updates. The monitor calls Binance REST independently of browser sessions. The repository provides no live stock data source; those values are demo/static.

4.2 Description of Modules and Key Features

Module	Verified responsibility
Frontend presentation	index.html and supporting pages; css/ and js/ modules for dashboard, charts, ideas, sign-in and sign-up.
API client and state	js/api.js centralizes requests and bearer-token headers; js/state.js maintains state and selected localStorage preferences.
Authentication	Register/login; bcryptjs hash/compare; JWT creation; protect middleware validates token and current tokenVersion.
Watchlist	Authenticated list/add/remove routes; each new record describes an asset. Legacy symbols[] documents remain readable.
Market data	Binance REST proxy and browser WebSockets for crypto; /status identifies static/mock stocks and indices.
Comments	Public GET by ideaId; authenticated POST and owner-only DELETE; content is trimmed and length limited.
Alerts	Authenticated CRUD; monitor default 30 seconds. Crypto only; stock alerts have no live data provider.
Notifications	Protected list, mark-read, mark-all-read, delete; monitor creates in-app notification on crypto trigger.
Persistence/configuration	Five explicit Mongoose collections. PORT, MONGODB_URI and JWT_SECRET are required; JWT_SECRET minimum 32 chars.

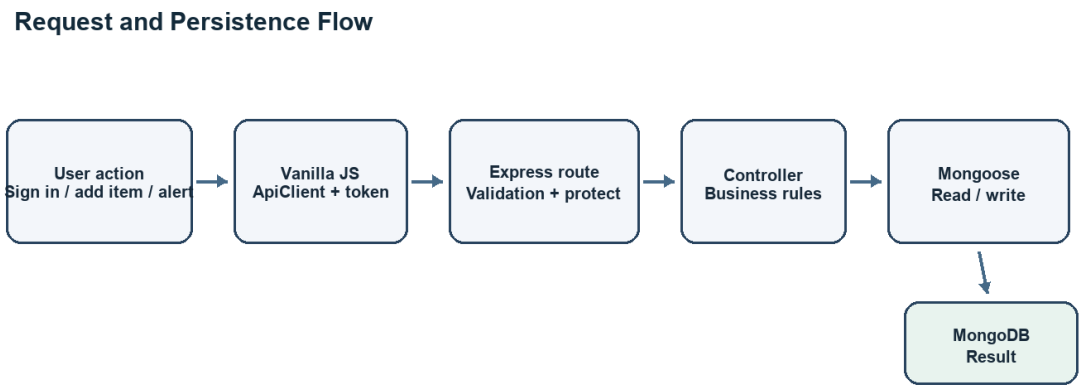


Figure 2. Typical authenticated request and persistence flow. Public market routes omit JWT protection.

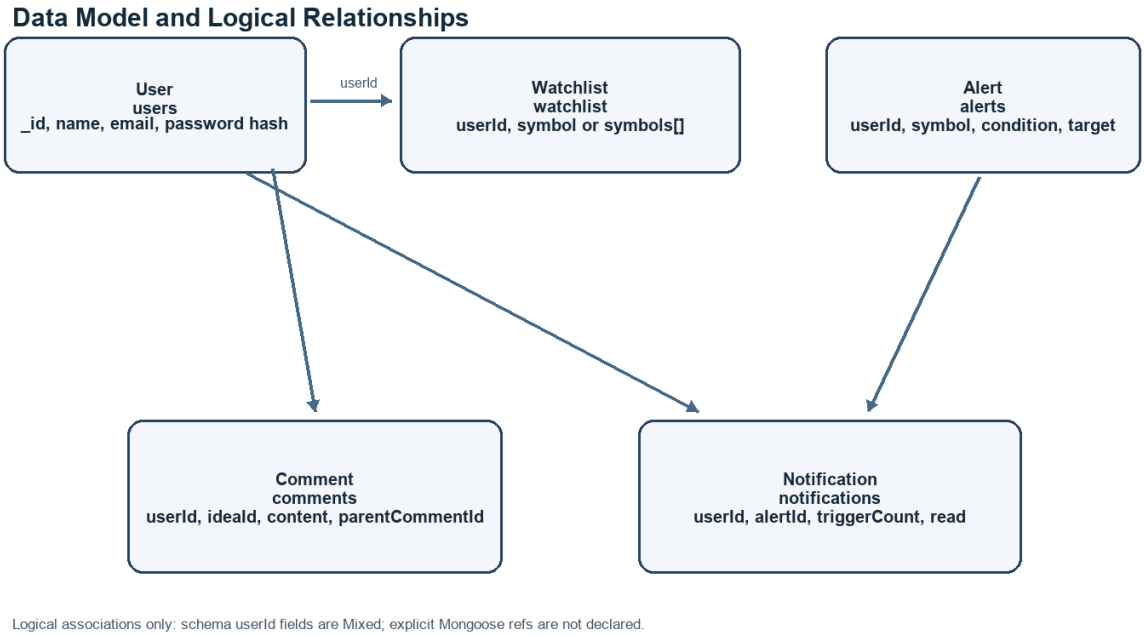


Figure 3. Collections and logical links. Mixed userId values have no explicit Mongoose refs.

Notification records include alertId and triggerCount; comments include userId, ideaId, and optional parentCommentId. Several ownership fields use Mixed values for string/ObjectId compatibility. These are logical links, not enforced foreign keys.

5. Implementation

The server loads dotenv, validates environment settings, installs CORS and body parsing, mounts routers, connects to MongoDB, starts the alert monitor outside test mode, and then listens on the configured port. Feature routers delegate to controllers. Mongoose models specify collection names and indexes. The frontend ApiClient wraps fetch calls and supplies authentication headers for protected resources.

5.1 Technologies Used

Technology	Role

HTML5, CSS3, Vanilla JavaScript	Page structure, layout, browser state, API requests, chart interaction and WebSockets. No React dependency was found.
Node.js	JavaScript runtime for backend ES modules.
Express.js 4.x	HTTP server, route mounting, middleware, REST endpoints; version range appears in package metadata.
MongoDB and Mongoose 8.x	Document database and object data modeling.
bcryptjs	Password hash generation and comparison.
jsonwebtoken (JWT)	Signed bearer token creation and verification.
Binance REST and WebSocket	Crypto historical candles, ticker and live streams; no order execution.
Lightweight Charts and Lucide	External browser libraries referenced in index.html for charting and icons.

5.2 Development Tools

Tool	Evidence status
Git	Verified: repository metadata and origin remote are present.
GitHub	Origin URL points to GitHub; active workflow usage is not established.
Node.js / npm	Backend package defines npm start and npm run dev; node_modules is present.
Visual Studio Code	Not confirmed by repository artifacts; list as used only if personally verified.
Postman / MongoDB Compass	Not confirmed. Possible manual verification tools, not claimed as used.
Terminal	Scripts are command-line compatible; individual usage cannot be inferred from source.

5.3 Sample Code Snippets

```
await connectDB();
if (process.env.NODE_ENV !== 'test') startAlertMonitor();
app.listen(config.port, () => {
  console.log(`[Express] Backend running on port ${config.port}`);
});
```

Source: backend/server.js. Credentials and private configuration are omitted.

```
const user = await User.findById(decoded.id).select('+tokenVersion');
if (!user || decoded.ver !== (user.tokenVersion || 0)) {
  return res.status(401).json({ success: false, message: 'Authentication session is no longer valid.' });
}
req.user = decoded;
return next();
```

Source: backend/middleware/authMiddleware.js. The user must still exist and token version must match.

```
const salt = await bcryptjs.genSalt(10);
const hashedPassword = await bcryptjs.hash(password, salt);
const newUser = new User({ name: displayName, username: displayName, email: cleanEmail, password:
hashedPassword });
await newUser.save();
const token = generateToken(newUser);
```

Source: backend/controllers/authController.js. The secret is never included.

```
const userSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, select: false },
  tokenVersion: { type: Number, default: 0, select: false }
}, { timestamps: false, collection: 'users' });
```

Source: backend/models/User.js (selected fields).

```
router.route('/')
  .get(protect, getWatchlist)
  .post(protect, addToWatchlist);
router.delete('/:id', protect, removeFromWatchlist);
```

Source: backend/routes/watchlistRoutes.js. Read/write endpoints require authentication.

```
if (assetType !== 'crypto') continue; // No live stock provider is configured.
const price = Number(prices.get(alert.symbol));
if (!Number.isFinite(price) || price <= 0) continue;
if (await triggerIfMatched(alert, price, previous)) triggeredCount += 1;
```

Source: backend/services/alertMonitor.js. Live monitoring explicitly skips stock alerts.

6. Results and Output

Source review confirms dashboard and sign-in/sign-up pages, browser modules for charts and market updates, Express routes for the listed feature groups, and Mongoose models for five collections. Backend scripts and dependencies are present. No server was launched, database connection attempted, API call executed, browser flow tested, or benchmark measured for this report. These are implementation findings rather than runtime test results.

6.1 Screenshots of the Working Project

No genuine runtime screenshots were available in the repository. Image assets are promotional backgrounds or mockup artwork and do not establish working application output. Replace each placeholder with a direct capture from the running application and add browser/date/test-database context. Do not use decorative assets as evidence of feature behavior.

Figure 4. Dashboard and market chart

SCREENSHOT PLACEHOLDER
Insert a direct browser screenshot showing the loaded dashboard, selected symbol, timeframe, and chart; state whether data is live or demo.

Figure 5. Sign in or sign up

SCREENSHOT PLACEHOLDER
Insert a form capture. Do not include real email addresses, passwords, JWTs, or personal information.

Figure 6. Watchlist

SCREENSHOT PLACEHOLDER
Insert an authenticated watchlist screenshot using a test account.

Figure 7. Idea comments

SCREENSHOT PLACEHOLDER
Insert an idea discussion with a test comment; redact account identifiers.

Figure 8. Price alerts and notifications

SCREENSHOT PLACEHOLDER
Insert alert setup and a triggered in-app notification; clarify only crypto alerts have live price monitoring.

Figure 9. API and database evidence

SCREENSHOT PLACEHOLDER
Insert sanitized API responses and a test-database collection view. Exclude secrets and personal data.

6.2 Key Outcomes

Category	Outcome
Implemented in source	Hashed-password auth/JWT validation; protected watchlist; comments; alert CRUD; in-app notification management; Binance REST proxy; crypto WebSocket client; periodic crypto alert monitor.
Partially implemented	Live coverage is crypto-only; stocks/indices are demo/static. Alerts monitor crypto only. Authenticated watchlists persist; unsigned state can use browser storage.
Not runtime-verified	Database connectivity, live Binance responses, login round trip, persistence, browser chart updates, alert trigger/recovery and API status codes.
Known limitations	No live stock provider, email/push sender, order execution, deployment or benchmark evidence; external market calls depend on network/provider availability.

7. Conclusion

The TradingView Clone is a modular web application prototype connecting a framework-free browser interface to a Node.js and Express REST backend, MongoDB persistence through Mongoose, JWT-protected account resources, and selected Binance crypto data. Code inspection supports account, watchlist, comments, price-alert, notification, and market-data modules. The work demonstrates separation of concerns, route and middleware design, schema-based persistence, password hashing, token validation, browser storage, HTTP APIs, and WebSocket streaming. Important scope limits remain: live market information is crypto-only, stock/index values are demo/static, and runtime verification must be completed before submission claims are finalized.

8. Future Scope

- **Broader market coverage:** Integrate a licensed equity data provider and label delayed, real-time, or simulated values.
- **Deployment:** Deploy with environment-specific CORS, TLS, managed MongoDB, health monitoring, and rollback.
- **Notification delivery:** Add opt-in email or push providers with retry policies, consent controls, and delivery status.
- **Security hardening:** Review rate limiting, validation, token storage/expiry, CSRF/XSS exposure, audit logs, dependency updates, and secret management.
- **Scalability and reliability:** Use a shared scheduler or queue for alerts, provider backoff, observability, caching, and load tests.
- **User experience:** Improve accessibility, responsive layout, chart errors, source labels, loading and offline indicators.
- **Automated verification:** Add API integration and browser end-to-end tests against isolated test data; record reproducible results.
- **Additional analytics:** Add chart indicators or providers after validating licensing, provenance, and symbol mapping.

9. Mapping to Program Outcomes (PO) and Sustainable Development Goals (SDG)

These mappings interpret the template examples for this project. Confirm final wording against the institution's approved outcomes.

Outcome	Project-specific justification
PO1: Computing fundamentals	JavaScript, HTTP/REST, asynchronous processing, document databases, schema modeling, and authentication.
PO2: Problem analysis	Translates fragmented monitoring needs into data display, user assets, discussion, and alerts; identifies demo equity data and untested paths as limitations.
PO3: Design and development of solutions	Separates frontend modules, routes, controllers, middleware, models, config, and alert service to implement the prototype.
PO5: Modern tool usage	Node.js, Express, MongoDB/Mongoose, Git, and Binance APIs are evidenced. VS Code, Postman, and Compass usage is unconfirmed.
PO9: Individual and team work	Not claimed: repository evidence does not establish whether work was individual or collaborative. Complete only after confirming arrangement.

Goal	Project-specific justification
SDG 9: Industry, Innovation and Infrastructure	Relevant as an educational software project exploring digital infrastructure and API integration. This is a learning contribution, not evidence of industrial deployment or measurable infrastructure impact.

10. References

[1] TradingView, "Supercharts: charting and analysis features," https://www.tradingview.com/features/ (accessed 18 October 2026).

[2] OpenJS Foundation, "Node.js Documentation," https://nodejs.org/en/docs/ (accessed 18 October 2026).

[3] OpenJS Foundation, "Express.js Guide: Routing," https://expressjs.com/guide/ (accessed 18 October 2026).

[4] MongoDB, Inc., "Mongoose Documentation: Schemas," https://mongoosejs.com/docs/guide.html (accessed 18 October 2026).

[5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015. https://datatracker.ietf.org/doc/html/rfc7519.

[6] D. T. Williams, "bcrypt.js," project documentation and source repository. https://github.com/dcodeIO/bcrypt.js (accessed 18 October 2026).

[7] Binance, "Binance Developer Documentation," https://developers.binance.com/en/docs/introduction (accessed 18 October 2026).

[8] Manassdev, "TradingViewClone," project repository. https://github.com/Manassdev/TradingViewClone (accessed 18 October 2026).

[9] Project source files reviewed: backend/server.js; backend/config/db.js; backend/config/env.js; backend/routes/*.js; backend/controllers/*.js; backend/middleware/*.js; backend/models/*.js; backend/services/alertMonitor.js; js/api.js; js/chart-engine.js; js/main.js; js/watchlist.js; backend/package.json.